

AWS Certified Generative AI Developer — Professional

The Use Case

- Financial services customer service chatbot on Bedrock, moving from pilot to full production
 - Challenges:** financial information must be factually accurate and current; responses must comply with regulation; sensitive data handled properly; consistent performance across customer segments; quality maintained through model updates
- The organising idea: traditional ML evaluation is insufficient for GenAI.** Traditional ML has one correct answer and measures accuracy, precision, recall. GenAI can have many valid answers, so evaluation shifts to **relevance, factual accuracy, consistency and fluency**.

1. Assessment Frameworks

The four evaluation dimensions

Dimension	Question it asks
Relevance	Does the answer address the question?
Factual accuracy	Are the facts correct?
Consistency	Does the same question get the same answer?
Fluency	Is the language clear and readable?

These are independent — an answer can be fluent but wrong, or accurate but irrelevant.

Transcript's financial dimensions:
factual accuracy of financial information
COMPLIANCE with financial regulations
consistency across similar queries
fluency and clarity of explanations
appropriateness of TONE for financial services

The last two are **domain-specific** — compliance and tone appear in no generic metric set.

LLM-as-a-Judge

Generator model -> produces the answer
Evaluator model -> scores the answer

These must be two different models. A model grading its own output introduces bias. The transcript uses Claude 3 Opus as judge for a separate production generator model.

- Results include **natural language explanations** alongside **normalised 0-1 scores**
- The explanation is what makes a score actionable — "3/5" alone tells you nothing you can fix
- Complete rubrics and judge prompts are **published in the documentation** for transparency
- Custom rubrics** per dimension are essential; without clear scoring criteria the evaluation becomes subjective

Bedrock Evaluations — four capabilities

Capability	Detail
LLM-as-a-judge	One model evaluates another with human-like assessment
Custom metrics	Your own criteria, numerical or categorical scoring
Environment-agnostic	Bring Your Own Inference — evaluate a model regardless of where it is hosted : Bedrock, SageMaker, your own servers, another cloud
Comprehensive assessment	Multiple dimensions at once

Datasets, human evaluation, weighted scoring

- Evaluation datasets must be **diverse and include edge cases** — you can only detect what your test set expresses (Task 3.4)
- Automatic and human evaluation complement each other** — automation gives scale, humans give ground truth
- Weighted scoring** combines dimensions into one overall score. In financial services, compliance carries far more weight than fluency.

2. Systematic Model Evaluation

A/B testing vs canary testing

Technique	Purpose	Mechanism
A/B testing	Comparison — which is better?	Both versions run; results compared statistically
Canary testing	Risk reduction — roll out safely	New version on a small traffic share, increased gradually

Transcript: A/B to compare temperature and top_p settings; canary to test model updates on a small percentage of traffic.

Multi-model, cost and latency analysis

- **Multi-model evaluation** — Claude, Llama and Titan compared for specific use cases
- **Cost-performance analysis** measures token efficiency to find the most cost-effective model **per query type** — not one model for everything (Task 4.1's tiered usage)
- **Latency-to-quality ratio** — is a marginal quality gain worth three extra seconds of customer waiting?
- **Business outcome metrics** connect model performance to organisational goals (Task 4.3)

3. User-Centered Evaluation

Two audiences, two kinds of feedback

Source	Tells you
Customers	Was it <i>helpful</i> ? — rating interface in the chatbot
Domain experts	Was it <i>correct</i> ? — annotation workflows in SageMaker Ground Truth

Customers often cannot tell that an answer is wrong — it reads confidently. Expert review is what catches factual and regulatory errors.

- **Kinesis Data Streams** captures user interaction patterns in real time at high volume
- **Rating systems** need separate scales per dimension — "helpful?" and "accurate?" are different questions
- **Sentiment analysis** extracts insight from unstructured feedback (Comprehend, as in Task 1.3)
- **Session analytics** reveal pain points — where users abandon, repeat themselves, or ask for a human

4. Quality Assurance Processes

- **Continuous evaluation with Step Functions** — the transcript runs tests against the production model **daily**
- **Regression testing** against a benchmark dataset of financial queries detects degradation (Tasks 1.6, 4.3)

Automated quality gates — CodePipeline

```
quality gate in the deployment pipeline
-> accuracy or compliance below threshold?
-> RELEASE BLOCKED
```

A substandard model cannot reach production, and the decision isn't left to human judgement in the moment. This is the enforcement mechanism behind the pipeline stages of Tasks 1.2 and 2.3.

Version control — including the datasets

Version **prompts and evaluation datasets** together. If the test set changed between runs, **v7 and v8 scores are not comparable** — the dataset must be versioned too.

Plus performance thresholds before deployment, production monitoring, alerting, and **rollback procedures** (Task 1.2's blue/green).

5. RAG Evaluation

Two evaluation types

Type	Evaluates
Retrieve only	Retrieval alone — did the right documents come back?
Retrieve and generate	The whole pipeline — retrieval plus the generated answer

Why both exist: when the pipeline produces a bad answer, you must know whether **retrieval failed or generation failed**. If retrieve-only passes but the final answer is wrong, the fault is in the model, not the retriever.

Retrieval quality metrics

Metric	Measures
Relevance scoring	How well retrieved information matches the query
Context matching	Whether retrieved content is appropriate to the situation — transcript: does this regulatory guidance actually apply to <i>this</i> case? A document can be relevant yet inapplicable.
Retrieval latency	Speed, especially for time-sensitive queries
Citation precision	Are the citations given correct ?
Citation coverage	Is everything that needed citing actually cited?

Also: test suites with **ground truth data, vector database evaluation** (embedding quality, similarity search performance), and **chunking strategy evaluation** — the systematic version of the trial-and-error tuning described in Task 1.5.

6. Agent Performance Frameworks

Metric	Assesses	Transcript example
Task completion rate	Did the agent finish the job?	Transfers, payments, account openings
Tool usage effectiveness	Right tool chosen and used well?	Interaction with backend financial systems
Reasoning quality	Logical progression through multi-step workflows	Loan applications, investment recommendations
Bedrock Agent evaluations	Specialised agent testing	Test cases across financial scenarios

Agent evaluation judges the process, not only the answer. An agent that reaches the right conclusion by an invalid route is still a problem — it will not generalise.

Also included: **adversarial testing** for edge cases and vulnerabilities (Task 3.1), and **ethical and responsible AI evaluation criteria** (Task 3.4).

7. Reporting Systems

QuickSight dashboards
automated reporting -> regular summaries
model comparison visualisations
trend analysis over time
drill-down for investigating specific issues
alert-based reporting for significant changes

Audience	Needs
Engineers	Detailed metrics, drill-down capability
Managers	Trends and comparisons
Executives	Business impact — technical metrics translated

As in Task 4.3: nobody outside engineering cares that latency fell to 1.8s. **"40% fewer factual errors" is the executive metric.**

8. Deployment Validation

Synthetic user workflows

Simulate **whole customer journeys**, not single queries — a customer asking five successive questions exercises the system very differently from one isolated prompt. (Task 2.5's synthetic monitoring, applied at deployment.)

AI-specific output validation

hallucination rates
semantic drift

Transcript: **semantic drift detection to ensure consistent interpretation of financial terminology**. If the new model understands "liquidity" differently, every downstream answer shifts.

Shadow deployment

run the new model on REAL production traffic
-> but DISCARD its output
-> users still receive the old model's answers
-> compare the two outputs offline

What users see	
Shadow	The old model — the new one runs only in the background
Canary	The new model, for a small share of users

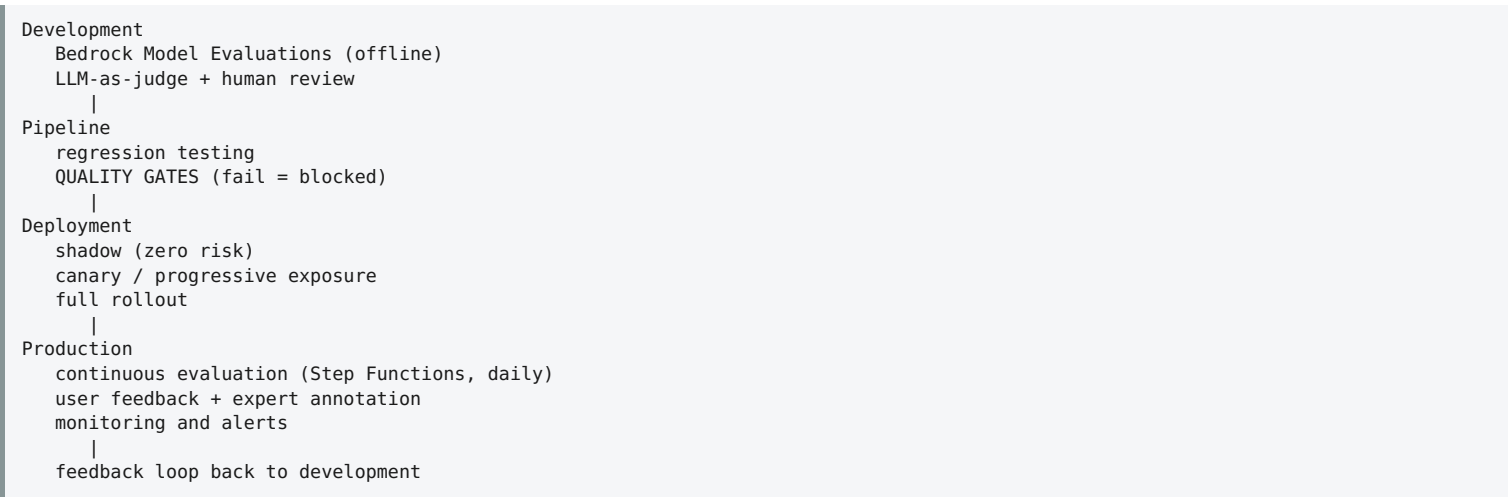
Shadow is the safest form of production testing — zero user risk, at the cost of running both models. The natural order is shadow → canary → progressive exposure → full rollout.

Deployment strategies compared

Strategy	Users see	Risk	Cost
Shadow	Old model	Zero	Double (both running)
Canary	New model, few users	Low	Normal
Progressive exposure	New model, growing share	Low, decreasing	Normal
Blue/Green	New model, everyone at once	Higher, but rollback is instant	Double (both environments)

Plus **automated fallback** reverting to the previous version when issues are detected (Task 3.3's automated remediation), and post-deployment monitoring (all of Domain 4).

The Full Evaluation Lifecycle



Reported outcomes

- 40% reduction in factual errors
- 99.8% compliance with financial regulations
- Consistent performance across customer segments; smooth updates with no disruption; higher satisfaction

The transcript's closing point is worth carrying forward: the evaluation system **also provides the documentation and evidence needed for regulatory compliance**. Evaluation is not only a quality practice — it is **governance evidence** (Task 3.3).